

Python String Manipulation

Unit 10.3: Python String functions(page 236)

1. What is a string?

A string is any sequence of characters wrapped in quotes. Even if a string looks like a number, once it comes from input(), it is treated as text — not as a number — unless you deliberately convert it.

```
tag = "1A2345Z"
tag = input("Enter the RFID tag: ") # always a string, even if digits
```

2. String length — len()

len() tells you how many characters are in a string. This is how you check that a string is a specific, required length.

```
len("1A2345Z") # 7
len("Hello")    # 5
len(tag) == 7   # checks tag is exactly 7 characters long
```

3. Indexing — getting a single character

Every character in a string has a position number called an index. Indexing starts at 0, not 1.

Tag	1	A	2	3	4	5	Z
Index	0	1	2	3	4	5	6

```
tag[0] # "1" -> first character
tag[1] # "A" -> second character
tag[6] # "Z" -> seventh character
tag[-1] # "Z" -> last character (negative index counts from the end)
```

⚠ Common mistake: using tag[7] for the "7th" character. It doesn't exist — since indexing starts at 0, the 7th character sits at index 6.

4. Slicing — getting a range of characters

Slicing pulls out a substring using string[start:stop]. The stop index is NOT included in the result.

```
tag = "1A2345Z"
tag[2:6] # "2345" -> characters at index 2,3,4,5 (index 6 is excluded)
```

Requirement	Code
1st character	tag[0]
2nd character	tag[1]
3rd-6th characters	tag[2:6]
7th (last) character	tag[6] or tag[-1]

5. Checking character type

Python strings have built-in methods that check what kind of characters they contain. Each returns True or False.

```
"5".isdigit() # True
"A".isalpha() # True -- is it a letter?
"z".isupper() # False -- is it uppercase?
"Z".isupper() # True
```

These methods also work on a whole substring at once, checking that every character in it matches:

```
"2345".isdigit()      # True  -- all 4 characters are digits
"23a5".isdigit()      # False -- one character is not a digit
```

This means a rule like "characters 3 to 6 must all be digits" can be checked in a single line:

```
tag[2:6].isdigit()
```

6. Combining conditions with and

An if statement can check several conditions at once using and. ALL conditions must be True for the whole expression to be True.

```
if condition1 and condition2 and condition3:
    print("All passed")
```

This matters for validation tasks: something should only be declared "valid" once every single rule has passed. Five separate, disconnected if statements that each print their own message do not satisfy this — everything needs to feed into one combined decision.

7. Recap table of syntax

Concept	Syntax	Example
Length	<code>len(s)</code>	<code>len("abc") -> 3</code>
Index (single char)	<code>s[i]</code>	<code>"abc"[1] -> "b"</code>
Slice (range)	<code>s[start:stop]</code>	<code>"abcdef"[1:4] -> "bcd"</code>
Is all digits	<code>s.isdigit()</code>	<code>"123".isdigit() -> True</code>
Is all letters	<code>s.isalpha()</code>	<code>"abc".isalpha() -> True</code>
Is uppercase	<code>s.isupper()</code>	<code>"AB".isupper() -> True</code>
Combine conditions	<code>and</code>	<code>a and b and c</code>

This example is not the assessment task — it uses the same shape of problem so students can practice applying the concepts above before attempting their own work.

Problem: validate a product code — 5 characters long, first 2 characters must be uppercase letters, last 3 characters must be digits.

```
code = input("Enter the product code: ")

if len(code) == 5 and code[0:2].isalpha() and code[0:2].isupper() and
code[2:5].isdigit():
    print("Product code is valid.")
else:
    print("Product code is invalid.")
```

Walk through each part of the if line as a class:

- `len(code) == 5` — is it the right length?
- `code[0:2].isalpha()` — are the first two characters letters?
- `code[0:2].isupper()` — are those letters uppercase?
- `code[2:5].isdigit()` — are the last three characters digits?

All four must be True together, or the whole condition is False.

Have students adapt this exact structure themselves to a different rule set as independent practice — same shape, different rules — before they touch their real assessment task.

Common mistakes to watch for

- Off-by-one errors with indexing (using index 7 for the "7th" character instead of 6)
- Confusing `.isupper()` (checks case) with `.isalpha()` (checks it's a letter at all)
- Writing separate unlinked if blocks instead of one combined condition — breaks the "all criteria together" logic
- Forgetting `input()` already returns a string — no need to `int()` convert unless doing arithmetic
- Slicing off-by-one — forgetting the stop index in `[start:stop]` is excluded